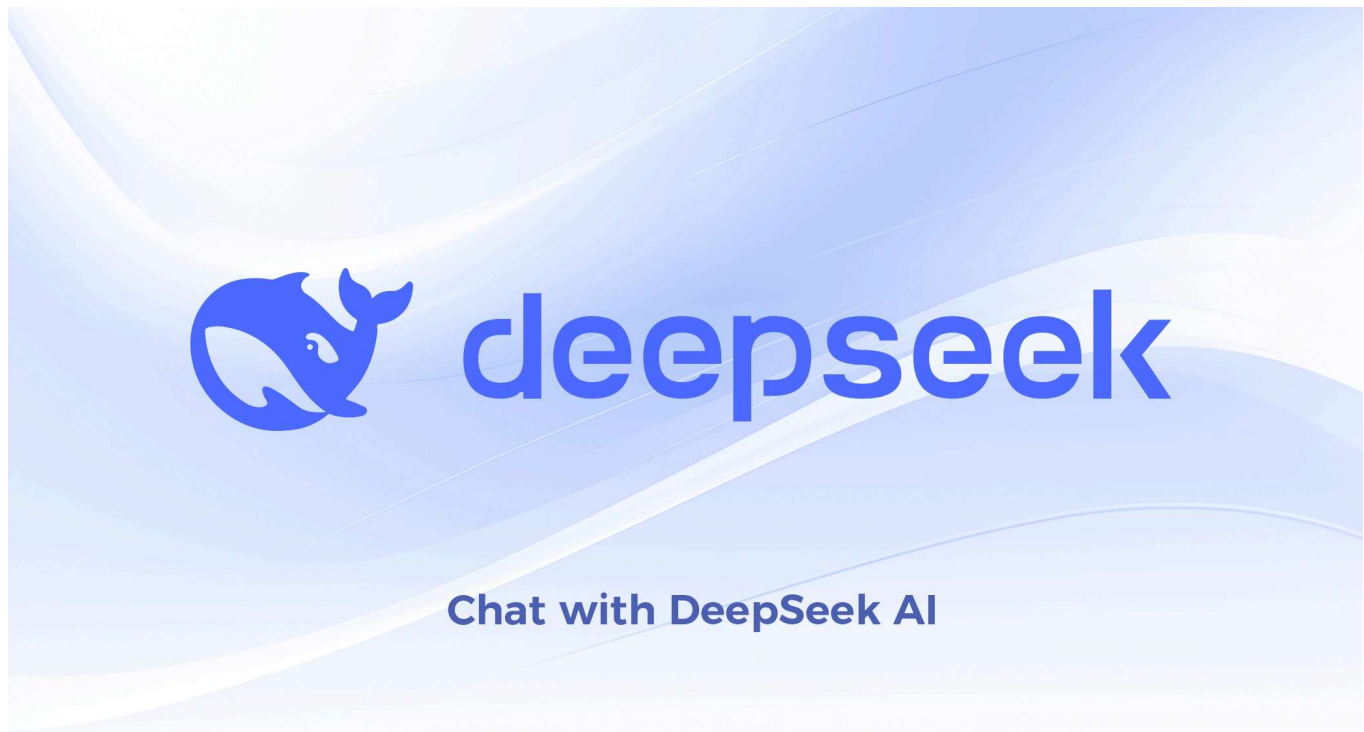


Python Inheritance Interview Guide - DeepSeek



COMPLETE OOPs CONCEPTS FOR INTERVIEW - All-in-One Guide

OOPs - Object Oriented Programming (Super Simple Definition)

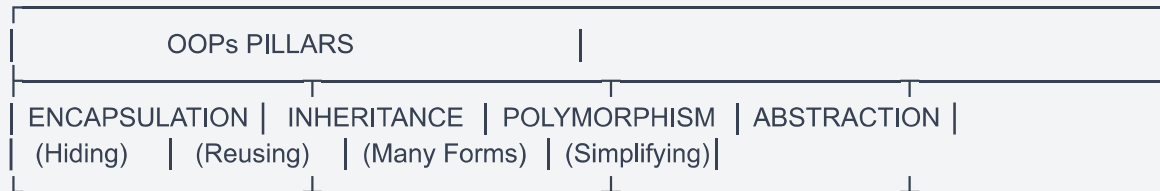
OOPs = A way of programming that organizes code into "objects" that contain both data and methods, just like real-world objects!

Think of OOPs like a Smartphone:

- **Object:** Your actual phone (iPhone, Samsung)
- **Class:** The blueprint/design of the phone
- **Data:** Photos, contacts, messages (stored inside)
- **Methods:** Camera, calling, texting (what it can do)

THE 4 PILLARS OF OOPs - Complete Coverage

text



1. CLASS AND OBJECT - The Foundation

What is a Class?

| Class = Blueprint/Template for creating objects

What is an Object?

| Object = Instance of a class (actual thing created from blueprint)

python

```
class Car:

    total_cars = 0

    def __init__(self, brand, model, year, price):

        self.brand = brand
        self.model = model
        self.year = year
        self.__price = price
        Car.total_cars += 1

    def start(self):
        return f"{self.brand} {self.model} is starting... Vroom! 🚗"

    def stop(self):
        return f"{self.brand} {self.model} stopped."

    def get_price(self):
        return f"${self.__price:,.2f}"

    def set_price(self, new_price):
        if new_price > 0:
            self.__price = new_price
            return "Price updated!"
        return "Invalid price!"

    def __str__(self):
        return f"{self.brand} {self.model} ({self.year})"

    @classmethod
    def get_total_cars(cls):
        return f"Total cars created: {cls.total_cars}"

    @staticmethod
    def is_motor_vehicle():
        return True

car1 = Car("Toyota", "Camry", 2024, 25000)
car2 = Car("Honda", "Civic", 2023, 22000)
car3 = Car("Tesla", "Model 3", 2024, 45000)
```

```
print(car1)
print(car1.start())
print(car1.get_price())
print(car2.get_price())
print(Car.get_total_cars())
print(Car.is_motor_vehicle())
```

2. ENCAPSULATION - "The Protective Shield"

Encapsulation = Wrapping data and methods together, and restricting direct access to protect the data

Real-Life Example:

- **ATM Machine:** Money inside (protected), you can only access through card and PIN
- **Medicine Capsule:** Powder inside, gelatin coating outside

python

```

class BankAccount:
    def __init__(self, account_holder, initial_balance):

        self.account_holder = account_holder
        self.account_number = self._generate_account_number()

        self._bank_name = "National Bank"

        self.__balance = initial_balance
        self.__pin = "1234"
        self.__transaction_history = []

    def deposit(self, amount):
        if self.__validate_amount(amount):
            self.__balance += amount
            self.__log_transaction("Deposit", amount)
            return f"✅ Deposited ${amount}. Balance: ${self.get_balance()}"
        return "❌ Invalid amount!"

    def withdraw(self, amount, pin):
        if not self.__verify_pin(pin):
            return "❌ Wrong PIN!"
        if amount > self.__balance:
            return "❌ Insufficient funds!"
        if amount <= 0:
            return "❌ Invalid amount!"

        self.__balance -= amount
        self.__log_transaction("Withdraw", amount)
        return f"✅ Withdrew ${amount}. Balance: ${self.get_balance()}"

    def get_balance(self):
        """Getter - read private data safely"""
        return f"${self.__balance:,.2f}"

    def get_transaction_history(self, pin):
        """Get history with PIN verification"""
        if self.__verify_pin(pin):
            return self.__transaction_history.copy()
        return "❌ Wrong PIN!"

    def __validate_amount(self, amount):
        return isinstance(amount, (int, float)) and amount > 0

    def __verify_pin(self, pin):
        return pin == self.__pin

```

```

def __log_transaction(self, type_, amount):
    from datetime import datetime
    self.__transaction_history.append({
        "type": type_,
        "amount": amount,
        "balance": self.__balance,
        "time": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    })

def _generate_account_number(self):
    """Protected method - for internal use"""
    import random
    return f"ACC{random.randint(10000, 99999)}"

account = BankAccount("Alice Johnson", 5000)

print(f"Holder: {account.account_holder}")
print(f"Account: {account.account_number}")

print(f"Bank: {account._bank_name}")

print(account.deposit(1000))
print(account.withdraw(500, "1234"))
print(account.get_balance())

history = account.get_transaction_history("1234")
for entry in history:
    print(f"{entry['time']}: {entry['type']} ${entry['amount']} (Balance: ${entry['balance']})")

```

3. INHERITANCE - "Like Parent, Like Child"

Inheritance = Child class automatically gets all properties and methods from Parent class

Types of Inheritance Complete Guide:

python

```
print("1 SINGLE INHERITANCE")
print("-" * 40)

class Employee:
    def __init__(self, name, id, salary):
        self.name = name
        self.id = id
        self.salary = salary

    def work(self):
        return f"{self.name} is working"

    def get_details(self):
        return f"ID: {self.id}, Name: {self.name}, Salary: ${self.salary}"

class Developer(Employee):
    def __init__(self, name, id, salary, programming_lang):
        super().__init__(name, id, salary)
        self.lang = programming_lang

    def work(self):
        return f"{self.name} is coding in {self.lang}"

    def debug(self):
        return f"{self.name} is debugging"

dev = Developer("Alice", "D001", 75000, "Python")
print(dev.get_details())
print(dev.work())
print(dev.debug())

print()
```

```
print("2 MULTILEVEL INHERITANCE")
print("-" * 40)

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def breathe(self):
        return f"{self.name} is breathing"
```

```
class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)
        self.student_id = student_id

    def study(self):
        return f"{self.name} is studying"

class CollegeStudent(Student):
    def __init__(self, name, age, student_id, major):
        super().__init__(name, age, student_id)
        self.major = major

    def attend_lecture(self):
        return f"{self.name} attending {self.major} lecture"

cs = CollegeStudent("Bob", 20, "ST123", "Computer Science")
print(cs.breathe())
print(cs.study())
print(cs.attend_lecture())

print()
```

```
print(" 3 HIERARCHICAL INHERITANCE")
print("-" * 40)
```

```
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        return f"{self.name} is eating"

    def sleep(self):
        return f"{self.name} is sleeping"

class Dog(Animal):
    def bark(self):
        return "Woof! Woof!"

class Cat(Animal):
    def meow(self):
        return "Meow! Meow!"

class Bird(Animal):
    def chirp(self):
        return "Chirp! Chirp!"
```



```
dog = Dog("Buddy")
cat = Cat("Whiskers")
bird = Bird("Tweety")

print(dog.eat())
print(cat.eat())
print(bird.eat())
print(dog.bark())
print(cat.meow())
print(bird.chirp())

print()

print("4 MULTIPLE INHERITANCE")
print("-" * 40)

class Father:
    def __init__(self):
        self.father_trait = "Tall"

    def skill(self):
        return "Cooking"

    def height(self):
        return "6 feet"

class Mother:
    def __init__(self):
        self.mother_trait = "Smart"

    def skill(self):
        return "Painting"

    def intelligence(self):
        return "High IQ"

class Child(Father, Mother):
    def __init__(self):
        Father.__init__(self)
        Mother.__init__(self)
        self.name = "Charlie"

    def skill(self):
        return "Programming"

    def get_all_traits(self):
        return f"{self.father_trait} and {self.mother_trait}"
```

```
child = Child()
print(f"Height: {child.height()}")
print(f"Intelligence: {child.intelligence()}")
print(f"Skill: {child.skill()}")
print(f"Traits: {child.get_all_traits()}")
print(f"MRO: {Child.__mro__}")
```

```
print()
```

```
print("5 HYBRID INHERITANCE")
print("-" * 40)
```

```
class Vehicle:
    def move(self):
        return "Moving..."
```

```
class Electric:
    def charge(self):
        return "Charging battery..."
```

```
class Car(Vehicle):
    def drive(self):
        return "Driving on road"
```

```
class ElectricCar(Car, Electric):
    def autopilot(self):
        return "Self-driving mode"
```

```
tesla = ElectricCar()
print(tesla.move())
print(tesla.drive())
print(tesla.charge())
print(tesla.autopilot())
```

4. POLYMORPHISM - "One Name, Many Forms"

Polymorphism = Same method name behaves differently for different objects

Types of Polymorphism:

python

```
print("1 METHOD OVERRIDING")
print("-" * 40)

class Shape:
    def area(self):
        return 0

    def perimeter(self):
        return 0

    def describe(self):
        return f"{self.__class__.__name__} - Area: {self.area():.2f}, Perimeter: {self.perimeter():.2f}"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        import math
        return math.pi * self.radius ** 2

    def perimeter(self):
        import math
        return 2 * math.pi * self.radius

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Triangle(Shape):
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def area(self):

        s = self.perimeter() / 2
        import math
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))
```

```
def perimeter(self):
    return self.a + self.b + self.c

shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 4, 5)]

for shape in shapes:
    print(shape.describe())

print()
```

```
print("2 DUCK TYPING")
print("-" * 40)
```

```
class Duck:
    def speak(self):
        return "Quack!"
    def fly(self):
        return "Duck flying"
```

```
class Airplane:
    def speak(self):
        return "Whoosh!"
    def fly(self):
        return "Airplane flying at 30,000 feet"
```

```
class Superman:
    def speak(self):
        return "I'm Superman!"
    def fly(self):
        return "Superman flying faster than a bullet!"
```

```
def make_it_fly_and_speak(thing):
    print(f"Speaking: {thing.speak()}")
    print(f"Flying: {thing.fly()}")
    print()
```

```
make_it_fly_and_speak(Duck())
make_it_fly_and_speak(Airplane())
make_it_fly_and_speak(Superman())

print()
```

```

print(" 3 OPERATOR OVERLOADING")
print("-" * 40)

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Point(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):
        return Point(self.x * scalar, self.y * scalar)

    def __truediv__(self, scalar):
        return Point(self.x / scalar, self.y / scalar)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __lt__(self, other):
        return (self.x**2 + self.y**2) < (other.x**2 + other.y**2)

    def __str__(self):
        return f"Point({self.x}, {self.y})"

p1 = Point(2, 3)
p2 = Point(4, 5)

print(f"p1 + p2 = {p1 + p2}")
print(f"p1 - p2 = {p1 - p2}")
print(f"p1 * 3 = {p1 * 3}")
print(f"p1 / 2 = {p1 / 2}")
print(f"p1 == p2: {p1 == p2}")
print(f"p1 < p2: {p1 < p2}")

print()

```

```

print(" 4 METHOD OVERLOADING (Simulated)")
print("-" * 40)

class Calculator:

    def add(self, a, b=0, c=0, d=0):
        return a + b + c + d

```

```

def multiply(self, *args):
    result = 1
    for num in args:
        result *= num
    return result

calc = Calculator()

print(f"add(10):    {calc.add(10)}")
print(f"add(10, 20): {calc.add(10, 20)}")
print(f"add(10,20,30): {calc.add(10, 20, 30)}")
print(f"multiply(2,3): {calc.multiply(2, 3)}")
print(f"multiply(2,3,4,5): {calc.multiply(2, 3, 4, 5)}")

print()

print("5 FUNCTION OVERLOADING with @singledispatch")
print("-" * 40)

from functools import singledispatch

@singledispatch
def process(data):
    print(f"Default: {data} (type: {type(data).__name__})")

@process.register(int)
def _(data):
    print(f"Integer: {data} (double: {data * 2})")

@process.register(str)
def _(data):
    print(f"String: '{data}' (length: {len(data)})")

@process.register(list)
def _(data):
    print(f"List: {data} ({len(data)} items)")

process(42)
process("Hello")
process([1, 2, 3])
process(3.14)

```

5. ABSTRACTION - "Hide Complexity, Show Simplicity"

Abstraction = Show only what's necessary, hide complex implementation details

python

```
from abc import ABC, abstractmethod

print("1 ABSTRACT CLASSES")
print("-" * 40)

class PaymentProcessor(ABC):
    def __init__(self, api_key):
        self.__api_key = api_key

    @abstractmethod
    def process_payment(self, amount):
        """Must be implemented by all payment processors"""
        pass

    @abstractmethod
    def refund(self, transaction_id):
        """Must be implemented by all payment processors"""
        pass

    def validate_amount(self, amount):
        return isinstance(amount, (int, float)) and amount > 0

    def get_processor_name(self):
        return self.__class__.__name__

class StripeProcessor(PaymentProcessor):
    def process_payment(self, amount):
        if not self.validate_amount(amount):
            return "Invalid amount"

        return f"✅ Stripe: Processed ${amount}"

    def refund(self, transaction_id):
        return f"💰 Stripe: Refunded {transaction_id}"

class PayPalProcessor(PaymentProcessor):
    def process_payment(self, amount):
        if not self.validate_amount(amount):
            return "Invalid amount"

        return f"✅ PayPal: Processed ${amount}"

    def refund(self, transaction_id):
```



```

        return f"💰 PayPal: Refunded {transaction_id}"

class CryptoProcessor(PaymentProcessor):
    def process_payment(self, amount):
        if not self.validate_amount(amount):
            return "Invalid amount"

        return f"✅ Crypto: Processed ${amount}"

    def refund(self, transaction_id):
        return f"💰 Crypto: Refunded {transaction_id}"

```

```

stripe = StripeProcessor("sk_live_123")
paypal = PayPalProcessor("pp_key_456")
crypto = CryptoProcessor("crypto_key_789")

```

```
processors = [stripe, paypal, crypto]
```

```

for processor in processors:
    print(f"\n{processor.get_processor_name():}")
    print(processor.process_payment(100))
    print(processor.refund("TXN123"))

```

```

print("\n\n🔹 DATABASE ABSTRACTION")
print("-" * 40)

```

```

class Database(ABC):
    @abstractmethod
    def connect(self):
        pass

    @abstractmethod
    def execute(self, query):
        pass

    @abstractmethod
    def close(self):
        pass

    def execute_transaction(self, queries):
        """Concrete method using abstract methods"""
        self.connect()
        results = []

```

```
        for query in queries:
            results.append(self.execute(query))
        self.close()
        return results

class MySQLDatabase(Database):
    def connect(self):
        return "MySQL: Connected"

    def execute(self, query):
        return f"MySQL: Executed '{query}'"

    def close(self):
        return "MySQL: Closed"

class PostgreSQLDatabase(Database):
    def connect(self):
        return "PostgreSQL: Connected"

    def execute(self, query):
        return f"PostgreSQL: Executed '{query}'"

    def close(self):
        return "PostgreSQL: Closed"

db = MySQLDatabase()
results = db.execute_transaction([
    "SELECT * FROM users",
    "INSERT INTO logs VALUES(1, 'test')"
])
for result in results:
    print(result)
```

6. SPECIAL METHODS (Magic/Dunder Methods)

python

```
class Book:
    def __init__(self, title, author, pages, price):
        self.title = title
        self.author = author
        self.pages = pages
        self.price = price

    def __str__(self):
        """For users - readable"""
        return f"{self.title} by {self.author}"

    def __repr__(self):
        """For developers - unambiguous"""
        return f"Book('{self.title}', '{self.author}', {self.pages}, {self.price})"

    def __add__(self, other):
        """Book + Book = combined pages"""
        return self.pages + other.pages

    def __mul__(self, quantity):
        """Book * quantity = total price"""
        return self.price * quantity

    def __eq__(self, other):
        """== based on title and author"""
        return self.title == other.title and self.author == other.author

    def __lt__(self, other):
        """< based on pages"""
        return self.pages < other.pages

    def __gt__(self, other):
        """> based on pages"""
        return self.pages > other.pages

    def __le__(self, other):
        """<= based on pages"""
        return self.pages <= other.pages

    def __len__(self):
        """len(book) returns pages"""
        return self.pages

    def __contains__(self, word):
        """word in book"""
        return word.lower() in self.title.lower()
```

```

def __getattr__(self, name):
    """Called when attribute not found"""
    return f"{name}' not found"

def __call__(self):
    """Make object callable like function"""
    return f"Reading '{self.title}'..."

def __enter__(self):
    print(f"Opening '{self.title}'...")
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    print(f"Closing '{self.title}'...")

book1 = Book("Python Mastery", "John Doe", 500, 49.99)
book2 = Book("Advanced Python", "Jane Smith", 350, 39.99)

print(f"str: {str(book1)}")
print(f"repr: {repr(book1)}")
print(f"len: {len(book1)} pages")
print(f"book1 + book2: {book1 + book2} pages")
print(f"book1 * 3: ${book1 * 3}")
print(f"book1 == book2: {book1 == book2}")
print(f"book1 > book2: {book1 > book2}")
print(f"'Python' in book1: {'Python' in book1}")
print(f"book1(): {book1()}")

with book1:
    print("Reading chapter 1...")

```

7. PROPERTY DECORATORS - Getters, Setters, Deleters

python

```
class Product:
    def __init__(self, name, price):
        self._name = name
        self._price = price
        self._discount = 0

    @property
    def price(self):
        """Get the current price"""
        return self._price

    @price.setter
    def price(self, value):
        """Set price with validation"""
        if value < 0:
            raise ValueError("Price cannot be negative!")
        self._price = value

    @price.deleter
    def price(self):
        """Reset price to 0"""
        self._price = 0
        print("Price reset to 0")

    @property
    def final_price(self):
        """Calculate final price after discount"""
        return self._price * (1 - self._discount / 100)

    @property
    def discount(self):
        return self._discount

    @discount.setter
    def discount(self, value):
        if 0 <= value <= 100:
            self._discount = value
        else:
            raise ValueError("Discount must be between 0 and 100")

    def __str__(self):
        return f"{self._name}: ${self._price} (Discount: {self._discount}%)"

product = Product("Laptop", 1000)
```

```
print(f"Original price: ${product.price}")
print(f"Final price: ${product.final_price}")

product.discount = 20
print(f"After 20% discount: ${product.final_price}")

product.price = 1200
print(f"New price: ${product.price}")

try:
    product.price = -100
except ValueError as e:
    print(f"Error: {e}")

del product.price
print(f"After deletion: ${product.price}")
```

8. CLASS METHODS vs STATIC METHODS vs INSTANCE METHODS

python

```
class Student:

    school = "Python University"
    total_students = 0

    def __init__(self, name, grade):

        self.name = name
        self.grade = grade
        Student.total_students += 1

    def get_info(self):
        """Access instance AND class data"""
        return f"{self.name} studies in grade {self.grade} at {self.school}"

    def update_grade(self, new_grade):
        """Modify instance data"""
        self.grade = new_grade
        return f"{self.name} moved to grade {new_grade}"

    @classmethod
    def get_school_info(cls):
        """Access class data only"""
        return f"School: {cls.school}, Total Students: {cls.total_students}"

    @classmethod
    def change_school(cls, new_school):
        """Modify class data"""
        cls.school = new_school
        return f"School changed to {new_school}"

    @classmethod
    def create_from_string(cls, student_string):
        """Alternative constructor"""
        name, grade = student_string.split('-')
        return cls(name, int(grade))

    @staticmethod
    def is_passing_grade(grade):
        """Doesn't need instance or class data"""
        return grade >= 60

    @staticmethod
    def calculate_percentage(marks, total):
        """Pure utility function"""
        return (marks / total) * 100 if total > 0 else 0
```

```
print("=" * 50)
print("INSTANCE, CLASS, AND STATIC METHODS")
print("=" * 50)

s1 = Student("Alice", 85)
s2 = Student("Bob", 45)

print(s1.get_info())
print(s2.get_info())
print(s1.update_grade(90))

print(Student.get_school_info())
print(Student.change_school("Advanced Python Academy"))
print(s1.get_info())

s3 = Student.create_from_string("Charlie-95")
print(s3.get_info())

print(f"Is 85 passing? {Student.is_passing_grade(85)}")
print(f"Is 45 passing? {Student.is_passing_grade(45)}")
print(f"Percentage: {Student.calculate_percentage(450, 500)}%")
```

9. COMPLETE REAL-WORLD SYSTEM - All Concepts Together

python


```
from abc import ABC, abstractmethod
from datetime import datetime

class User(ABC):
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.__password = None
        self.created_at = datetime.now()

    def set_password(self, password):
        if len(password) >= 8:
            self.__password = self._hash_password(password)
            return "Password set successfully"
        return "Password too short!"

    def _hash_password(self, password):
        """Protected method - simple hash for demo"""
        return f"hashed_{password}"

    def verify_password(self, password):
        return self.__password == self._hash_password(password)

    @abstractmethod
    def get_role(self):
        """ABSTRACTION - must implement"""
        pass

    @abstractmethod
    def get_permissions(self):
        """ABSTRACTION - must implement"""
        pass

class Admin(User):
    def get_role(self):
        return "Administrator"

    def get_permissions(self):
        return ["read", "write", "delete", "manage_users"]

    def manage_system(self):
        return "Managing system settings"

class Editor(User):
    def __init__(self, username, email, department):
        super().__init__(username, email)
        self.department = department
```

```

def get_role(self):
    return "Editor"

def get_permissions(self):
    return ["read", "write", "edit_content"]

def edit_article(self, article_id):
    return f"Editing article {article_id}"

class Viewer(User):
    def get_role(self):
        return "Viewer"

    def get_permissions(self):
        return ["read"]

class UserProfile:
    def __init__(self, user, bio="", avatar=""):
        self.user = user
        self.bio = bio
        self.avatar = avatar
        self.followers = []
        self.posts = []

    def add_post(self, content):
        if "write" in self.user.get_permissions():
            self.posts.append({
                "content": content,
                "author": self.user.username,
                "timestamp": datetime.now()
            })
            return "Post added!"
        return "Permission denied!"

    def __str__(self):
        return f"Profile: {self.user.username} ({self.user.get_role()})"

print("=" * 60)
print("COMPLETE SYSTEM - ALL OOPs CONCEPTS")
print("=" * 60)

admin = Admin("admin_john", "john@admin.com")
editor = Editor("editor_jane", "jane@editor.com", "Tech")
viewer = Viewer("viewer_bob", "bob@viewer.com")

print(admin.set_password("admin12345"))

```

```

print(editor.set_password("editor123"))
print(viewer.set_password("viewer123"))

users = [admin, editor, viewer]

print("\n 🚦 User Permissions:")
for user in users:
    print(f'{user.username} ({user.get_role()}) : {user.get_permissions()}')

profiles = {
    "admin": UserProfile(admin, "System Administrator"),
    "editor": UserProfile(editor, "Tech Content Editor"),
    "viewer": UserProfile(viewer, "Regular Reader")
}

print("\n 🧪 Testing Content Creation:")

print(f'Admin: {profiles["admin"].add_post("System maintenance scheduled")}')

print(f'Editor: {profiles["editor"].add_post("New Python tutorial")}')

print(f'Viewer: {profiles["viewer"].add_post("My post")}')

print("\n 👤 User Profiles:")
for profile in profiles.values():
    print(profile)
    if profile.posts:
        print(f"  Posts: {len(profile.posts)}")
        for post in profile.posts:
            print(f"    - {post['content']}")

print("\n 🛡️ Security Check:")
print(f'Admin password correct: {admin.verify_password("admin12345")}')
print(f'Admin password wrong: {admin.verify_password("wrongpass")}')

```

10. INTERVIEW QUICK REFERENCE

Key Definitions:

Common Interview Questions:

python

```
class Example:
    def __str__(self):
        return "User-friendly string"
```

```
    def __repr__(self):
        return "Example()"
```

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
print(D.__mro__)
```

```
class Demo:
    @classmethod
    def class_method(cls):
        pass

    @staticmethod
    def static_method():
        pass
```

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class ImmutablePoint:
    x: int
    y: int
```

```
class LoggingMixin:
    def log(self, message):
        print(f"[{self.__class__.__name__}] {message}")
```

```
class MyClass(LoggingMixin):
    def do_something(self):
        self.log("Doing something...")
```

11. DO's AND DON'Ts MASTER LIST

DO:

python

```
class Car:
    def __init__(self):
        self.engine = Engine()
```

```
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    @property
    def balance(self):
        return self.__balance
```

```
class Child(Parent):
    def __init__(self):
        super().__init__()
```

✗ DON'T:

python

```
class Stack(list):  
    pass  
  
class Stack:  
    def __init__(self):  
        self._items = []  
  
class A: pass  
class B(A): pass  
class C(B): pass  
class D(C): pass  
  
class Bad:  
    def __init__(self):  
        self.secret_data = "exposed"  
  
class D(A, B, C, D, E):  
    pass  
  
class BadChild(ABC):  
    pass
```

12. COMPLETE OOPs CHEAT SHEET

text

OOPs INTERVIEW CHEAT SHEET

4 PILLARS:

- └─ Encapsulation → Hide data (`__private`, `__protected`)
- └─ Inheritance → Reuse code (`class Child(Parent)`)
- └─ Polymorphism → One name, many forms (overriding, duck typing)
- └─ Abstraction → Hide complexity (ABC, `@abstractmethod`)

KEY CONCEPTS:

- └─ Class → Blueprint (`class MyClass`)
- └─ Object → Instance (`obj = MyClass()`)
- └─ Constructor → `__init__(self)`
- └─ `self` → Refers to instance
- └─ `super()` → Call parent class
- └─ `cls` → Refers to class (`@classmethod`)
- └─ MRO → Method Resolution Order
- └─ Magic Methods → `__str__`, `__repr__`, `__add__`, etc.

RELATIONSHIPS:

- └─ IS-A → Inheritance (Dog IS-A Animal)
- └─ HAS-A → Composition (Car HAS-A Engine)
- └─ USES-A → Dependency

IMPORTANT DECORATORS:

- └─ `@property` → Getter
- └─ `@name.setter` → Setter
- └─ `@classmethod` → Class method
- └─ `@staticmethod` → Static method
- └─ `@abstractmethod` → Abstract method

BEST PRACTICES:

- └─ Keep classes small and focused
- └─ Use composition over inheritance
- └─ Make attributes private by default
- └─ Use `@property` for controlled access
- └─ Follow SOLID principles
- └─ Write docstrings for classes and methods

Remember This Mantra:

"OOPs is about creating objects that contain both data and methods, organized around the 4 pillars: Encapsulation (hiding), Inheritance (reusing), Polymorphism (flexibility), and Abstraction (simplifying)!"

Quick Mental Model:

- **Class** = Blueprint

- **Object** = Actual thing
- **Encapsulation** = Private box
- **Inheritance** = Parent → Child
- **Polymorphism** = Many forms
- **Abstraction** = Simple interface

This covers everything you need to ace your OOPs interview! 🚀

oops all concept and its very easy meaning to understand everyone [properties methods and complex word] and its short ans like def and code also

OOPs COMPLETE GUIDE - Super Easy Definitions & Code

What is OOPs? (Easiest Definition)

OOPs = Programming by creating "objects" that have properties (data) and methods (actions), just like real-world things!

Think of a Mobile Phone:

- **Properties (Data/Attributes):** color, brand, battery, storage
- **Methods (Actions/Functions):** call(), click_photo(), charge()
- **Object:** Your actual phone (iPhone 15, Red color, 128GB)

ALL OOPs CONCEPTS - Simple Words

text

OOPs = Organizing code like real-world objects

- |
- 1. CLASS & OBJECT
- 2. PROPERTIES & METHODS
- 3. ENCAPSULATION (Hiding)
- 4. INHERITANCE (Reusing)
- 5. POLYMORPHISM (Many Forms)
- 6. ABSTRACTION (Simplifying)
- 7. CONSTRUCTOR
- 8. SPECIAL METHODS

1. CLASS AND OBJECT

Class = Design/Blueprint

Object = Actual thing made from blueprint

python

```
class House:
    pass

my_house = House()
your_house = House()

print(type(my_house))
```

Simple Example:

python

```
class Car:

    brand = "Toyota"
    color = "Red"
    price = 25000

    def start(self):
        return "Car started! 🚗"

    def stop(self):
        return "Car stopped!"

    def honk(self):
        return "Beep Beep! 🚦"

car1 = Car()
car2 = Car()

print(car1.brand)
print(car1.start())
print(car1.honk())

car2.color = "Blue"
print(car1.color)
print(car2.color)
```

2. PROPERTIES AND METHODS

Properties = Data/Attributes (WHAT object HAS)

Methods = Functions/Actions (WHAT object DOES)

python

```
class Student:

    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade
        self.marks = 0

    def study(self, subject):
        return f"{self.name} is studying {subject} 📖"

    def give_exam(self):
        return f"{self.name} is giving exam 📝"

    def update_marks(self, marks):
        self.marks = marks
        return f"Marks updated to {marks}%"

    def get_report(self):
        return f"{self.name} (Age: {self.age}) - Grade: {self.grade}, Marks: {self.marks}%"

alice = Student("Alice", 15, "10th")
bob = Student("Bob", 16, "11th")

print(alice.name)
print(bob.age)

print(alice.study("Math"))
print(bob.give_exam())
print(alice.update_marks(95))
print(alice.get_report())
```

3. ENCAPSULATION = "Hiding Data"

Simple Meaning:

Like a medicine capsule - powder inside (hidden), coating outside (visible)

Why?

- Protect data from outside

- Control how data is accessed

python

```
class BankAccount:
    def __init__(self, name, initial_money):
        self.name = name
        self.__money = initial_money
        self.__pin = "1234"

    def check_balance(self, pin):
        """Only show balance if PIN is correct"""
        if pin == self.__pin:
            return f"Balance: ${self.__money}"
        return "❌ Wrong PIN!"

    def deposit(self, amount):
        """Add money with validation"""
        if amount > 0:
            self.__money += amount
            return f"✅ Added ${amount}. New balance: ${self.__money}"
        return "❌ Invalid amount!"

    def withdraw(self, amount, pin):
        """Remove money with PIN verification"""
        if pin != self.__pin:
            return "❌ Wrong PIN!"
        if amount > self.__money:
            return "❌ Not enough money!"
        if amount <= 0:
            return "❌ Invalid amount!"

        self.__money -= amount
        return f"✅ Withdrew ${amount}. Remaining: ${self.__money}"

acc = BankAccount("John", 1000)

print(acc.name)

print(acc.deposit(500))
print(acc.check_balance("1234"))
print(acc.withdraw(200, "1234"))
print(acc.check_balance("0000"))
```

Public vs Protected vs Private:

python

```
class Demo:
    def __init__(self):
        self.public = "Anyone can see me! 👁️ "
        self._protected = "Please don't touch me! ⚠️ "
        self.__private = "Completely hidden! 🔒 "

obj = Demo()

print(obj.public)
print(obj._protected)
```

4. INHERITANCE = "Getting from Parents"

Simple Meaning:

| Child gets everything from parent automatically, like you inherit your father's house

Types of Inheritance:

python

```
print("1. SINGLE INHERITANCE")
```

```
class Animal:
    def eat(self):
        return "Eating... 🍖 "

    def sleep(self):
        return "Sleeping... 😴 "

class Dog(Animal):
    def bark(self):
        return "Woof Woof! 🐶 "

dog = Dog()
print(dog.eat())
print(dog.sleep())
print(dog.bark())
```

```
print("\n2. MULTILEVEL INHERITANCE")
```

```
class Grandparent:
    def property(self):
        return "House and Land 🏠 "

class Parent(Grandparent):
    def business(self):
        return "Shop 🏪 "

class Child(Parent):
    def education(self):
        return "Engineering Degree 🎓 "

child = Child()
print(child.property())
print(child.business())
print(child.education())
```

```
print("\n3. HIERARCHICAL INHERITANCE")
```

```
class Vehicle:
    def move(self):
```

```
        return "Moving... 🚗"

class Car(Vehicle):
    def drive(self):
        return "Driving on road 🚗"

class Boat(Vehicle):
    def sail(self):
        return "Sailing on water 🚤"

class Plane(Vehicle):
    def fly(self):
        return "Flying in air ✈️"

car = Car()
boat = Boat()
plane = Plane()

print(car.move())
print(boat.move())
print(plane.move())
print(car.drive())
print(boat.sail())
print(plane.fly())

print("\n4. MULTIPLE INHERITANCE")

class Father:
    def height(self):
        return "Tall (6 feet) 📏"

    def skill(self):
        return "Cooking 🍳"

class Mother:
    def intelligence(self):
        return "Very Smart 🧠"

    def skill(self):
        return "Painting 🎨"

class Child(Father, Mother):
    def hobby(self):
        return "Playing Guitar 🎸"

child = Child()
print(child.height())
print(child.intelligence())
```



```
print(child.skill())
print(child.hobby())

print("\n5. USING super()")

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print(f"Person created: {name}")

    def introduce(self):
        return f"Hi, I'm {self.name}"

class Student(Person):
    def __init__(self, name, age, grade):
        super().__init__(name, age)
        self.grade = grade
        print(f"Student grade: {grade}")

    def introduce(self):

        return super().introduce() + f", Grade: {self.grade}"

student = Student("Alice", 20, "A")
print(student.introduce())
```

5. POLYMORPHISM = "Many Forms"

Simple Meaning:

Same action works differently for different things - like "Play" button on TV vs Music Player

Types of Polymorphism:

python

```
print("1. METHOD OVERRIDING")
print("-" * 30)
```

```
class Shape:
    def area(self):
        return 0

    def info(self):
        return f"I am a shape"

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h
```

```
shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(f"Area: {shape.area()}")
```

```
print("\n2. DUCK TYPING")
print("-" * 30)
```

```
class Duck:
    def sound(self):
        return "Quack! 🦆 "
```

```
class Dog:
    def sound(self):
        return "Woof! 🐕 "
```

```
class Cat:
    def sound(self):
        return "Meow! 🐈 "
```

```
def make_sound(animal):
    print(animal.sound())

make_sound(Duck())
make_sound(Dog())
make_sound(Cat())


print("\n3. OPERATOR OVERLOADING")
print("-" * 30)

class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __sub__(self, other):
        return Money(self.amount - other.amount)

    def __gt__(self, other):
        return self.amount > other.amount

    def __str__(self):
        return f"${self.amount}"

m1 = Money(100)
m2 = Money(50)

print(f"{m1} + {m2} = {m1 + m2}")
print(f"{m1} - {m2} = {m1 - m2}")
print(f"{m1} > {m2}: {m1 > m2}")
```

```
print("\n4. FUNCTION OVERLOADING")
print("-" * 30)

class Calculator:

    def add(self, a, b=0, c=0):
        return a + b + c

    def multiply(self, *args):
```

```
result = 1
for n in args:
    result *= n
return result

calc = Calculator()
print(f"add(5): {calc.add(5)}")
print(f"add(5,3): {calc.add(5,3)}")
print(f"add(5,3,2): {calc.add(5,3,2)}")
print(f"multiply(2,3): {calc.multiply(2,3)}")
print(f"multiply(2,3,4): {calc.multiply(2,3,4)}")
```

6. ABSTRACTION = "Hide Complexity"

Simple Meaning:

| Like TV remote - you just press button, don't need to know complex circuits inside

python

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount):
        """Every payment method MUST have pay()"""
        pass

    @abstractmethod
    def refund(self, amount):
        """Every payment method MUST have refund()"""
        pass

    def receipt(self, amount):
        return f"Receipt: ${amount} paid"

class CreditCard(Payment):
    def pay(self, amount):
        return f"💳 Paid ${amount} via Credit Card"

    def refund(self, amount):
        return f"💰 Refunded ${amount} to Credit Card"

class PayPal(Payment):
    def pay(self, amount):
        return f"💳 Paid ${amount} via PayPal"

    def refund(self, amount):
        return f"💰 Refunded ${amount} to PayPal"

credit = CreditCard()
paypal = PayPal()

print(credit.pay(100))
print(paypal.pay(200))
print(credit.receipt(100))
```

```
class CoffeeMachine(ABC):
    @abstractmethod
    def make_coffee(self):
        pass

class SimpleCoffeeMachine(CoffeeMachine):
    def make_coffee(self):

        self.__boil_water()
        self.__brew()
        return "☕ Coffee ready!"

    def __boil_water(self):
        print("Boiling water...")

    def __brew(self):
        print("Brewing coffee...")

machine = SimpleCoffeeMachine()
print(machine.make_coffee())
```

7. CONSTRUCTOR = "Object Creator"

Simple Meaning:

| **Constructor = Special method that runs automatically when you create an object**

python

```
class Person:

    def __init__(self, name, age):
        self.name = name
        self.age = age
        print(f"🎉 New person created: {name}")

    def greet(self):
        return f"Hi, I'm {self.name}, {self.age} years old"

p1 = Person("Alice", 25)
p2 = Person("Bob", 30)

print(p1.greet())
print(p2.greet())


class Simple:
    pass

obj = Simple()


class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

s1 = Student("Alice", "A")


class Logger:
    def __init__(self):
        self.logs = []
        print("Logger started!")

log = Logger()


class Book:
    def __init__(self, title="Unknown", author="Unknown", pages=0):
        self.title = title
        self.author = author
        self.pages = pages
```

```
b1 = Book()  
b2 = Book("Python 101")  
b3 = Book("Python 101", "John", 300)
```

8. SPECIAL METHODS (Magic/Dunder Methods)

Simple Meaning:

Special methods with double underscore (__) that make objects work like built-in types

python


```
class Book:
    def __init__(self, title, author, pages, price):
        self.title = title
        self.author = author
        self.pages = pages
        self.price = price

    def __str__(self):
        """For print() - readable"""
        return f"{self.title} by {self.author}"

    def __repr__(self):
        """For developers - detailed"""
        return f"Book('{self.title}', '{self.author}', {self.pages}, {self.price})"

    def __len__(self):
        """len() function"""
        return self.pages

    def __add__(self, other):
        """+ operator"""
        return self.price + other.price

    def __sub__(self, other):
        """- operator"""
        return self.price - other.price

    def __mul__(self, quantity):
        """* operator"""
        return self.price * quantity

    def __eq__(self, other):
        """== operator"""
        return self.title == other.title

    def __lt__(self, other):
        """< operator - compare by pages"""
        return self.pages < other.pages

    def __gt__(self, other):
        """> operator"""
        return self.pages > other.pages

    def __getitem__(self, key):
        """book['key']"""
```

```

    if key == 'title':
        return self.title
    elif key == 'author':
        return self.author
    return None

def __contains__(self, word):
    """'word' in book"""
    return word.lower() in self.title.lower()

def __call__(self):
    """Make object callable like function"""
    return f"Reading '{self.title}'... 📖"

def __enter__(self):
    print(f"Opening '{self.title}'...")
    return self

def __exit__(self, *args):
    print(f"Closing '{self.title}'...")

b1 = Book("Python Mastery", "John Doe", 500, 49.99)
b2 = Book("Advanced Python", "Jane Smith", 350, 39.99)

print(f"str: {str(b1)}")
print(f"repr: {repr(b1)}")
print(f"len: {len(b1)} pages")
print(f"b1 + b2: ${b1 + b2}")
print(f"b1 - b2: ${b1 - b2}")
print(f"b1 * 3: ${b1 * 3}")
print(f"b1 == b2: {b1 == b2}")
print(f"b1 > b2: {b1 > b2}")
print(f"b1['title']: {b1['title']}")
print(f"'Python' in b1: {'Python' in b1}")
print(f"Call: {b1()}")

with b1 as book:
    print("Reading chapter 1...")

```

9. PROPERTY DECORATORS - Smart Attributes

Simple Meaning:

Make attributes smart - they can validate, calculate, or protect data

python

```
class Product:
    def __init__(self, name, price):
        self._name = name
        self._price = price
        self._discount = 0

    @property
    def price(self):
        """Get price"""
        return self._price

    @price.setter
    def price(self, value):
        """Set price with validation"""
        if value < 0:
            raise ValueError("Price cannot be negative!")
        self._price = value

    @price.deleter
    def price(self):
        """Delete price (reset to 0)"""
        self._price = 0
        print("Price reset!")

    @property
    def final_price(self):
        """Calculate final price automatically"""
        return self._price * (1 - self._discount / 100)

    @property
    def discount(self):
        return self._discount

    @discount.setter
    def discount(self, value):
        if 0 <= value <= 100:
            self._discount = value
        else:
            raise ValueError("Discount must be 0-100%")

    def __str__(self):
        return f"{self._name}: ${self.price} (-{self.discount}% = ${self.final_price})"

product = Product("Laptop", 1000)
```

```
print(product)

product.discount = 20
print(product)

product.price = 1200
print(product)


try:
    product.price = -100
except ValueError as e:
    print(f"Error: {e}")
```

10. CLASS METHOD vs STATIC METHOD vs INSTANCE METHOD

python

```
class Student:
    school = "Python Academy"
    count = 0

    def __init__(self, name, grade):
        self.name = name
        self.grade = grade
        Student.count += 1

    def introduce(self):
        return f"I'm {self.name}, Grade: {self.grade}"

    def update_grade(self, new_grade):
        self.grade = new_grade

    @classmethod
    def get_school_info(cls):
        return f"School: {cls.school}, Students: {cls.count}"

    @classmethod
    def change_school(cls, new_name):
        cls.school = new_name

    @classmethod
    def create_from_string(cls, text):
        """Alternative constructor"""
        name, grade = text.split('-')
        return cls(name, int(grade))

    @staticmethod
    def is_passing(grade):
        return grade >= 60

    @staticmethod
    def calculate_percentage(marks, total):
        return (marks/total)*100 if total > 0 else 0

s1 = Student("Alice", 85)
s2 = Student("Bob", 45)

print(s1.introduce())
print(s2.introduce())

print(Student.get_school_info())
```

```
Student.change_school("Advanced Python")
print(Student.get_school_info())

s3 = Student.create_from_string("Charlie-95")
print(s3.introduce())

print(Student.is_passing(85))
print(Student.is_passing(45))
print(Student.calculate_percentage(450, 500))
```

11. COMPLETE REAL EXAMPLE - All Concepts Together

python

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name, id, base_salary):

        self.name = name
        self.id = id
        self.__base_salary = base_salary
        self.__bonus = 0

    @abstractmethod
    def calculate_salary(self):
        pass

    @abstractmethod
    def get_role(self):
        pass

    def set_bonus(self, amount):
        if amount > 0:
            self.__bonus = amount

    def get_base_salary(self):
        return self.__base_salary

    def get_bonus(self):
        return self.__bonus

    def get_info(self):
        return f"{self.name} ({self.id}) - {self.get_role()}"

class Manager(Employee):
    def __init__(self, name, id, base_salary, team_size):
        super().__init__(name, id, base_salary)
        self.team_size = team_size

    def calculate_salary(self):
        return self.get_base_salary() + self.get_bonus() + (self.team_size * 500)

    def get_role(self):
        return "Manager"

class Developer(Employee):
```



```
def __init__(self, name, id, base_salary, skills):
    super().__init__(name, id, base_salary)
    self.skills = skills

def calculate_salary(self):
    skill_bonus = len(self.skills) * 1000
    return self.get_base_salary() + self.get_bonus() + skill_bonus

def get_role(self):
    return "Developer"

print("=" * 50)
print("EMPLOYEE MANAGEMENT SYSTEM")
print("=" * 50)

employees = [
    Manager("Alice", "M001", 80000, 5),
    Developer("Bob", "D001", 65000, ["Python", "Java", "React"]),
    Developer("Charlie", "D002", 70000, ["Python", "Django", "AWS", "Docker"])
]

employees[0].set_bonus(5000)
employees[1].set_bonus(3000)
employees[2].set_bonus(4000)

for emp in employees:
    salary = emp.calculate_salary()
    print(f"{emp.get_info():30} | Salary: ${salary:>8,}")

class Salary:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Salary(self.amount + other.amount)

    def __str__(self):
        return f"${self.amount:}"

s1 = Salary(50000)
s2 = Salary(30000)
print(f"\nTotal Salary: {s1 + s2}")
```

12. ULTIMATE CHEAT SHEET

text

OOPs QUICK REFERENCE

📦 CLASS & OBJECT

- Class = Design (Blueprint)
- Object = Actual thing (Instance)

🏰 4 PILLARS

- Encapsulation = Hide & Protect data
 - Public: `self.name`
 - Protected: `self._data` (convention)
 - Private: `self.__data` (enforced)
- Inheritance = Reuse from parent
 - Single: `class Child(Parent)`
 - Multilevel: `Parent → Child → Grandchild`
 - Multiple: `class Child(P1, P2)`
 - `super()`: Call parent method
- Polymorphism = Same name, different behavior
 - Overriding: Child changes parent method
 - Duck Typing: If it has method, use it
 - Overloading: Same name, different params
- Abstraction = Hide complexity
 - ABC: Abstract Base Class
 - `@abstractmethod`: Must implement in child

🔧 SPECIAL METHODS

- `__init__` = Constructor
- `__str__` = String for users
- `__repr__` = String for developers
- `__len__` = `len()` function
- `__add__` = `+` operator
- `__eq__` = `==` operator
- `__lt__` = `<` operator
- `__call__` = Make object callable

🎯 METHOD TYPES

- Instance: `def method(self)` - Uses object data
- Class: `@classmethod` - Uses class data
- Static: `@staticmethod` - Utility function

💡 PROPERTIES

- `@property` - Getter (read)
- `@name.setter` - Setter (write with validation)
- `@name.deleter` - Deleter (cleanup)



REMEMBER THIS MANTRA:

text

CLASS = Blueprint (Design)
OBJECT = Actual thing (Instance)
PROPERTY = What object HAS (Data)
METHOD = What object DOES (Action)

ENCAPSULATION = Hide data (Private box)
INHERITANCE = Reuse code (Parent→Child)
POLYMORPHISM = Many forms (Same name, different work)
ABSTRACTION = Hide complexity (Simple interface)

Quick Memory Trick:

"C O P I E" - Class, Object, Properties, Inheritance, Encapsulation

"A P P" - Abstraction, Polymorphism, Properties/Methods

This is **ALL** you need to understand OOPs! 🚀